Aubrée Romain **Beauvallet**
10/03/2026 **Semestre 2**
SLAM

Groupe 1

Bloc 2 TD-7

Contexte 2
Exercice 1.1 2
Exercice 1.2 2
Exercice 1.3-5 3
Exercice 1.6 4
Exercice 1.7 4
Exercice 1.8 5
Exercice 1.9 5
Exercice 1.10-12 6
Exercice 1.13-14 7
Exercice 1.15 8
Exercice 2.1 8
Exercice 2.2 9
Exercice 2.3 10
Exercice 2.4 10
Exercice 2.5 10
Exercice 2.6 11
Exercice 2.7 12
Exercice 2.8 12
Exercice 2.9 13
Exercice 2.10 13
Exercice 2.11 13
Exercice 2.12 14
Exercice 2.13 14
Exercice 2.14 16
Conclusion 17

Contexte

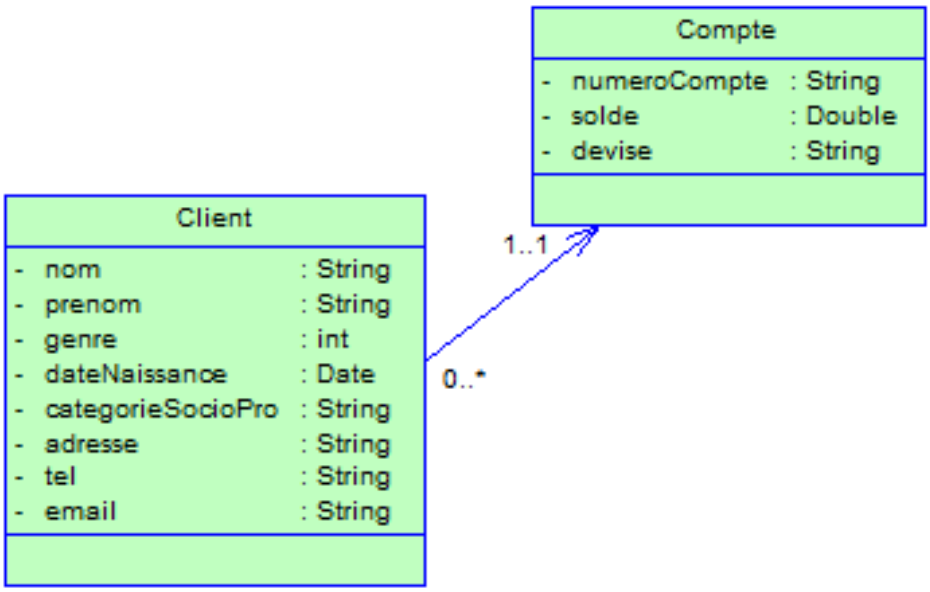
Cours et création d'un programme de gestion de banque en utilisant l'héritage de classe. (Le TP contient le projet IntelliJ et le fichier PowerAMC disponible sur mon portfolio).

Exercice 1.1

L'héritage permet de créer des classes à partir de classes existantes.

Exemple : Classe mère **Vehicule** (attributs : vitesse, poids). Classes enfants **Voiture** (ajoute nbPortes) et Moto (ajoute typeGuidon).

Exercice 1.2



Exercice 1.3-5

```
CLASSE Compte
  PROTEGE numero : Chaine
  PROTEGE titulaire : Chaine
  PROTEGE solde : Réel
  PROTEGE devise : Chaine

  METHODE Constructeur(p_num: Chaine, p_tit: Chaine, p_solde: Réel, p_dev: Chaine)
    numero <- p_num
    titulaire <- p_tit
    solde <- p_solde
    devise <- p_dev
  FIN METHODE

  METHODE getSolde() RETOURNE Réel : RETOURNE solde
  METHODE setSolde(p_solde: Réel) : solde <- p_solde

  METHODE debiter(montant: Réel) : solde <- solde - montant
  METHODE crediter(montant: Réel) : solde <- solde + montant

  METHODE Decrire() RETOURNE Chaine
    RETOURNE "n°: " + numero + " - solde : " + solde + " " + devise
  FIN METHODE
FIN CLASSE
```

Exercice 1.6

```
METHODE info_comptes()
  AFFICHER "Liste des comptes de M. " + nom + " " + prenom
  POUR CHAQUE cpt DANS listeComptes FAIRE
    AFFICHER cpt.Decrire()
  FIN POUR
FIN METHODE
```

Exercice 1.7

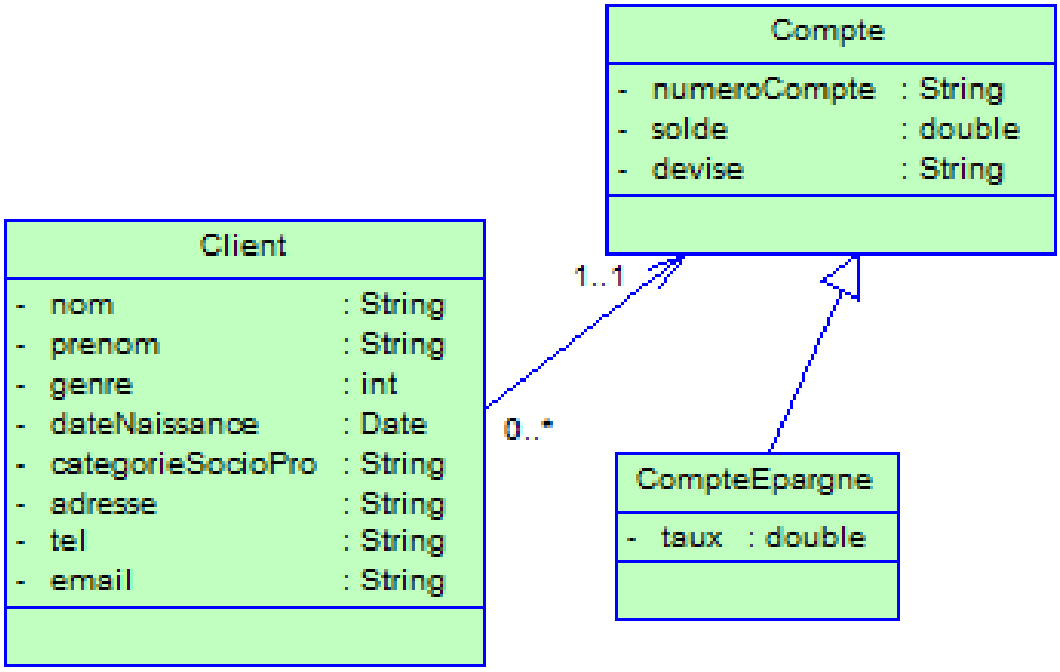
```
ALGO Principale_Q1_7
  Client1 <- NOUVEAU Client("Martin", "Benjamin", 1, "02/05/1977", "enseignant", "5 boulevard AB, 06000 Nice", "01.01.01.01.01", "martinG@gmail.com")

  Cpt1 <- NOUVEAU Compte("13245411324", "Martin Benjamin", 2548200, "€")
  Cpt2 <- NOUVEAU Compte("32544771289", "Martin Benjamin", 1000, "€")
  Cpt3 <- NOUVEAU Compte("22544771289", "Martin Benjamin", -214, "€")

  Client1.ajouterCompte(Cpt1)
  Client1.ajouterCompte(Cpt2)
  Client1.ajouterCompte(Cpt3)

  Client1.info_comptes()
FIN
```

Exercice 1.8

Identiques : numero, titulaire, solde, devise.

Différents : taux_interet.

Exercice 1.9



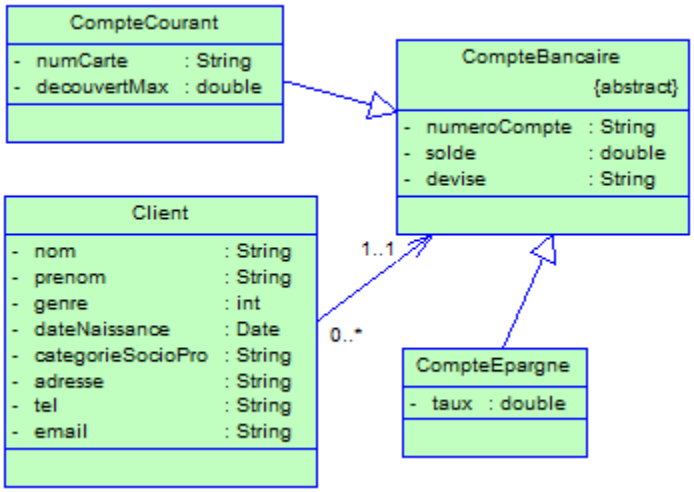
Exercice 1.10-12

```
CLASSE CompteEpargne HERITE DE Compte
  PRIVER tauxInteret : Réel

  METHODE Constructeur(p_num: Chaine, p_tit: Chaine, p_solde: Réel, p_dev: Chaine, p_taux: Réel)
    SUPER(p_num, p_tit, p_solde, p_dev)
    tauxInteret <- p_taux
  FIN MÉTHODE

  METHODE Decrire() RETOURNE Chaine
    RETOURNE "Compte Epargne n°: " + numero + " - solde: " + solde + " " + devise + " - taux = " + (tauxInteret * 100) + "%"
  FIN METHODE

  METHODE ajouter_interet()
    SI solde > 0 ALORS
      crediter(solde * tauxInteret)
    FIN SI
  FIN METHODE
FIN CLASSE
```

Exercice 1.13-14

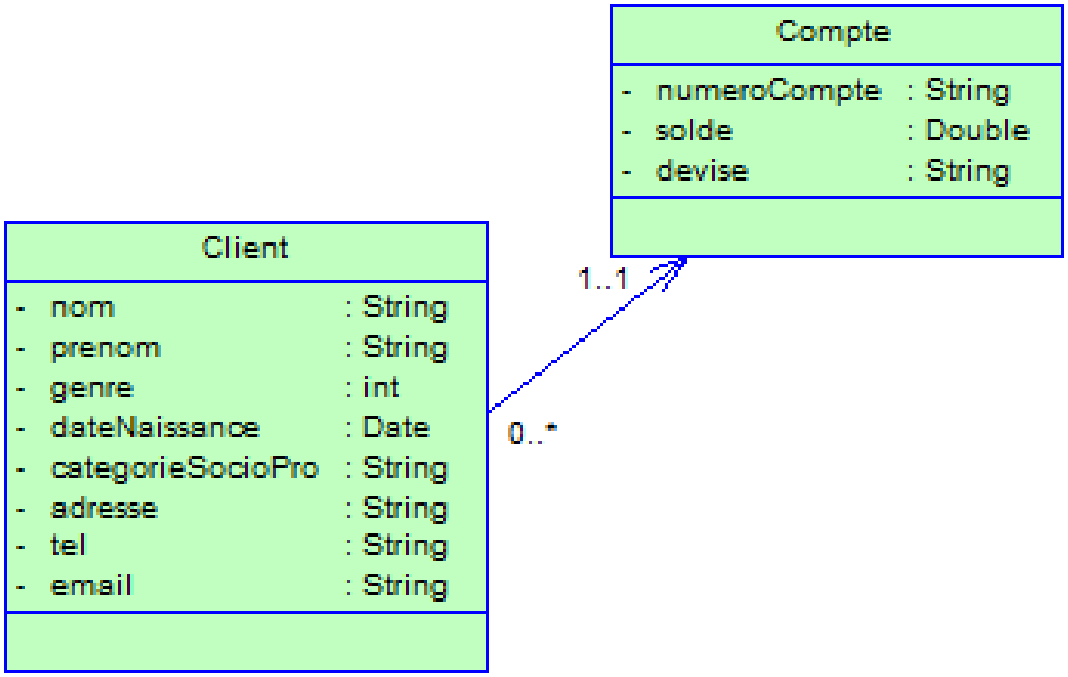
```
ALGO Principal_Q1_13_14

   Client2 <- NOUVEAU Client("Martin", "Benjamin", 1, "02/05/1977", "enseignant", "5
boulevard AB 06000 Nice", "01.01.01.01.01", "gravouilG@gmail.com")


   Cpt1 <- NOUVEAU Compte("13245411324", "Martin Benjamin", 2548200, "€")

   CptEpargne <- NOUVEAU CompteEpargne("32544771289", "Martin Benjamin", 1000, "€",
0.05)

   Cpt3 <- NOUVEAU Compte("22544771289", "Martin Benjamin", -214, "€")


   CptEpargne.ajouter_interet()


   Client2.ajouterCompte(Cpt1)

   Client2.ajouterCompte(CptEpargne)

   Client2.ajouterCompte(Cpt3)


   Client2.info_comptes()

FIN
```

Exercice 1.15



Exercice 2.1

Fait avec PowerAMC :



Exercice 2.2

(Code complet disponible sous forme de projet)

```java
public Client(String nom, String prenom, int genre, Date dateNaissance,  no usages
              String categorieSocioPro, String adresseComplete, String telephone, String email) {
    this.nom = nom;
    this.prenom = prenom;
    this.genre = genre;
    this.dateNaissance = dateNaissance;
    this.categorieSocioPro = categorieSocioPro;
    this.adresseComplete = adresseComplete;
    this.telephone = telephone;
    this.email = email;
    this.comptes = new ArrayList<>();
}

public String getNom() { return nom; }  no usages
public void setNom(String nom) { this.nom = nom; }  no usages

public String getPrenom() { return prenom; }  no usages
public void setPrenom(String prenom) { this.prenom = prenom; }  no usages

public int getGenre() { return genre; }  no usages
public void setGenre(int genre) { this.genre = genre; }  no usages

public Date getDateNaissance() { return dateNaissance; }  no usages
public void setDateNaissance(Date dateNaissance) { this.dateNaissance = dateNaissance; }  no usages

public String getCategorieSocioPro() { return categorieSocioPro; }  no usages
public void setCategorieSocioPro(String categorieSocioPro) { this.categorieSocioPro = categorieSocioPro; }

public String getAdresseComplete() { return adresseComplete; }  no usages
public void setAdresseComplete(String adresseComplete) { this.adresseComplete = adresseComplete; }  no usage

public String getTelephone() { return telephone; }  no usages
public void setTelephone(String telephone) { this.telephone = telephone; }  no usages

public String getEmail() { return email; }  no usages
public void setEmail(String email) { this.email = email; }  no usages

public List<Compte> getComptes() { return comptes; }  no usages
public void setComptes(List<Compte> comptes) { this.comptes = comptes; }  no usages
```

```java
package org.banquetd7.td7banque;

public class Compte {
    private String numero;  3 usages
    private double solde;  3 usages
    private String devise;  3 usages
    private Client titulaire;  3 usages

    public Compte(String numero, double solde, String devise, Client titulaire) {
        this.numero = numero;
        this.solde = solde;
        this.devise = devise;
        this.titulaire = titulaire;
    }

    public String getNumero() { return numero; }  no usages
    public void setNumero(String numero) { this.numero = numero; }  no usages

    public double getSolde() { return solde; }  no usages
    public void setSolde(double solde) { this.solde = solde; }  no usages

    public String getDevise() { return devise; }  no usages
    public void setDevise(String devise) { this.devise = devise; }  no usages

    public Client getTitulaire() { return titulaire; }  no usages
    public void setTitulaire(Client titulaire) { this.titulaire = titulaire; }  no
}
```

Exercice 2.3

On empêche l'utilisateur de débiter 0 ou moins

```
public void crediter(double montant) {  no usages
    if (montant <= 0) {
        System.out.println("Le montant doit être supérieur a 0.");
        return;
    }
    this.solde += montant;
}

public void debiter(double montant) {  no usages
    if (montant <= 0) {
        System.out.println("Le montant doit être supérieur a 0.");
        return;
    }
    this.solde -= montant;
}
```

Exercice 2.4

```
public String decrire() {  no usages
    return "n°: " + this.numeroCompte + " - solde: " + this.solde + " " + this.devise;
}
```

Exercice 2.5

```
public void info_comptes() {  no usages
    String civilite = (this.genre == 1) ? "M." : "Mme";

    System.out.println("Liste des comptes de " + civilite + " " + this.nom + " " + this.prenom);

    for (Compte compteActuel : this.comptes) {
        System.out.println(compteActuel.decrire());
    }
}
```

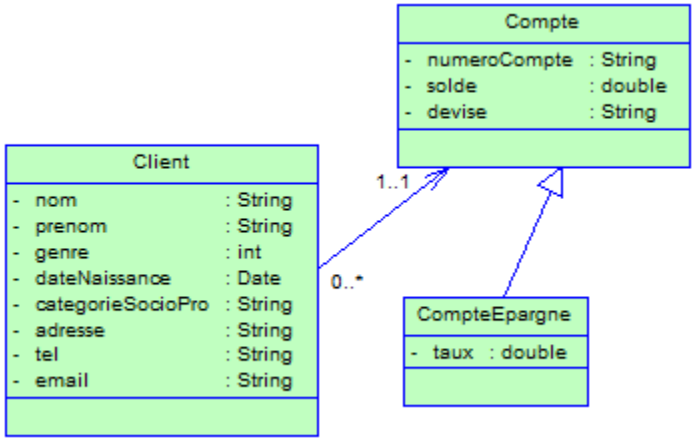
Exercice 2.6

```java
package org.banquetd7.td7banque;

import java.time.LocalDate;

public class Main {
    public static void main(String[] args) {
        Client benjamin = new Client(
                nom: "Gravouil", prenom: "Benjamin", genre: 1, LocalDate.of( year: 1977, month: 5, dayOfMonth: 2),
                categorieSocioPro: "enseignant", adresse: "lycée Paul Lapie, 5 boulevard AB, 92400 COURBEVOIE",
                tel: "01.01.01.01.01", email: "prof.gravouil@gmail.com");

        Compte c1 = new Compte( numeroCompte: "13245411324", solde: 2548200, devise: "€", benjamin);
        Compte c2 = new Compte( numeroCompte: "32544771289", solde: 1000, devise: "€", benjamin);
        Compte c3 = new Compte( numeroCompte: "22544771289", solde: -214, devise: "€", benjamin);

        benjamin.getComptes().add(c1);
        benjamin.getComptes().add(c2);
        benjamin.getComptes().add(c3);

        System.out.println("--- Affichage initial ---");
        benjamin.info_comptes();

        c3.crediter( montant: 100);

        System.out.println("\n--- Après crédit de 100€ sur le compte c3 ---");
        benjamin.info_comptes();
    }
}
```

```
--- Affichage initial ---
Liste des comptes de M. Gravouil Benjamin
n°: 13245411324 - solde: 2548200.0 €
n°: 32544771289 - solde: 1000.0 €
n°: 22544771289 - solde: -214.0 €

--- Après crédit de 100€ sur le compte c3 ---
Liste des comptes de M. Gravouil Benjamin
n°: 13245411324 - solde: 2548200.0 €
n°: 32544771289 - solde: 1000.0 €
n°: 22544771289 - solde: -114.0 €
```

Exercice 2.7

Ajout de **CompteEpargne** qui hérite de Compte.



Exercice 2.8

“extends” permet de récupérer les caractéristiques de l’autre classe donc pas besoin de réécrire les variables.

```
package org.banquetd7.td7banque;

public class CompteEpargne extends Compte {
    private double taux;  3 usages

    public CompteEpargne(String numero, double solde, String devise, Client titulaire, double taux) {
        super(numero, solde, devise, titulaire);
        this.taux = taux;
    }
    public double getTaux() {  no usages
        return taux;
    }
    public void setTaux(double taux) {  no usages
        this.taux = taux;
    }
}
```

Exercice 2.9

On multiplie le taux par 100 pour avoir la valeur en pourcentage.

```
@Override  2 usages
public String decrire() {
    return "Compte Epargne " + super.decrire() + " - taux = " + (int)(this.getTaux() * 100) + "%";
}
```

Exercice 2.10

Permet d'ajouter les intérêts sur le compte épargne.

```
public void ajouterInteret() {  no usages
    if (this.getSolde() > 0) {
        double calculInteret = this.getSolde() * this.taux;
        this.crediter(calculInteret);
    }
}
```

Exercice 2.11

Le 2éme compte a été modifié pour être un compte épargne.

```
Compte c1 = new Compte( numeroCompte: "13245411324",  solde: 2548200,  devise: "€", benjamin);
CompteEpargne c2 = new CompteEpargne( numeroCompte: "32544771289",  solde: 1000,  devise: "€", benjamin,  taux: 0.05);
Compte c3 = new Compte( numeroCompte: "22544771289",  solde: -214,  devise: "€", benjamin);
```

```
Liste des comptes de M. Gravouil Benjamin
n°: 13245411324 - solde: 2548200.0 €
Compte Epargne n°: 32544771289 - solde: 1000.0 € - taux = 5%
n°: 22544771289 - solde: -214.0 €

--- Après crédit de 100€ sur le compte c3 ---
Liste des comptes de M. Gravouil Benjamin
n°: 13245411324 - solde: 2548200.0 €
Compte Epargne n°: 32544771289 - solde: 1000.0 € - taux = 5%
n°: 22544771289 - solde: -114.0 €
```

Exercice 2.12

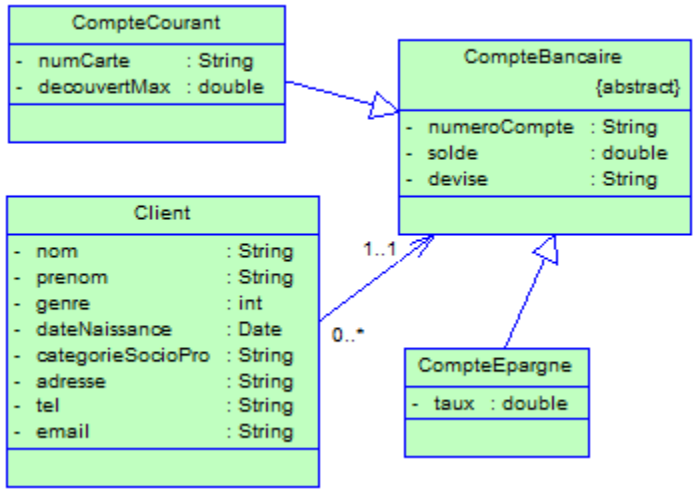
On appel les fonctions déjà faites

```
System.out.println("\n--- Ajout d'intêret sur le compte c3 ---");
c2.ajouterInteret();

System.out.println("\n--- Après ajout des intérêts sur le compte épargne ---");
benjamin.info_comptes();
```

```
n°: 13245411324 - solde: 2548200.0 €
Compte Epargne n°: 32544771289 - solde: 1000.0 € - taux = 5%
n°: 22544771289 - solde: -114.0 €

--- Ajout d'intêret sur le compte c3 ---

--- Après ajout des intérêts sur le compte épargne ---
Liste des comptes de M. Gravouil Benjamin
n°: 13245411324 - solde: 2548200.0 €
Compte Epargne n°: 32544771289 - solde: 1050.0 € - taux = 5%
n°: 22544771289 - solde: -114.0 €
```

Exercice 2.13

On renomme **“Compte”** -> **“CompteBancaire”** pour tout relié, et on ajoute **“CompteCourant”** qui est l’équivalent de l’ancien **“Compte”**



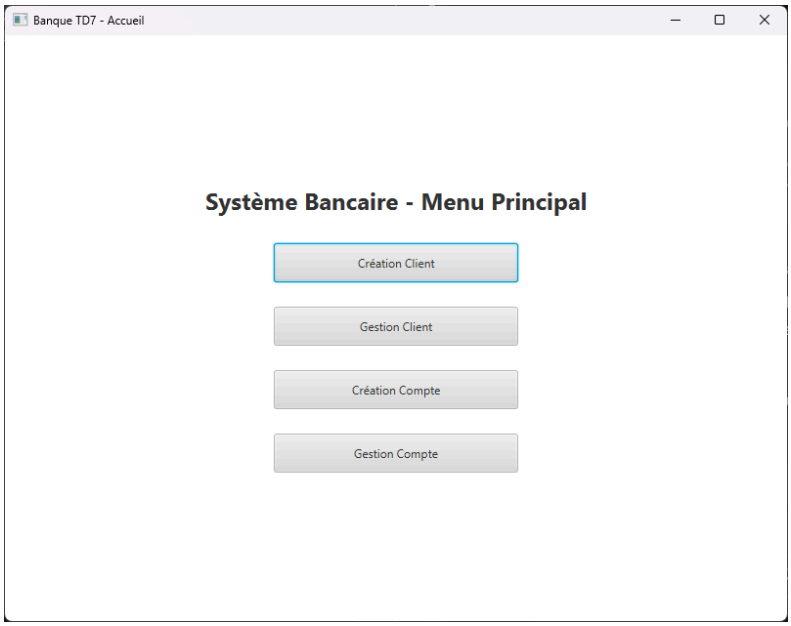

```
public abstract class CompteBancaire
```

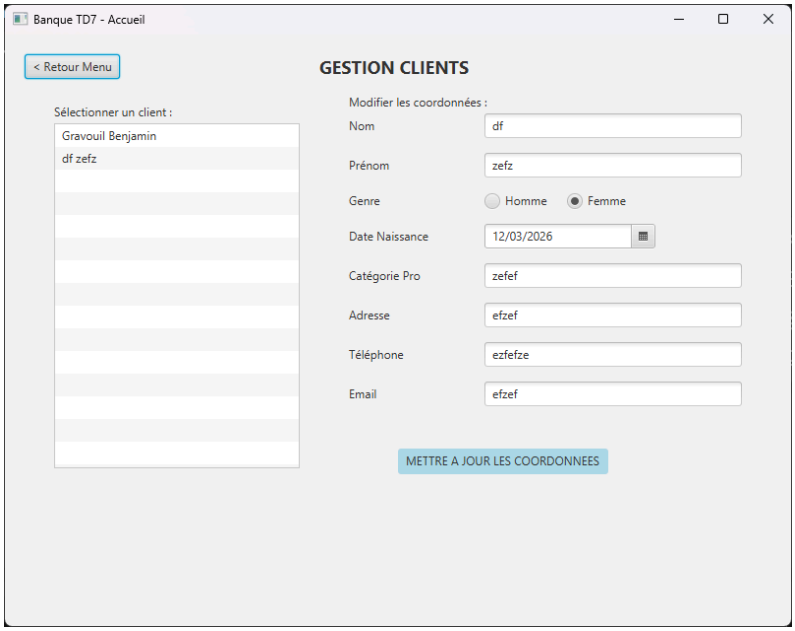
```java
public class CompteCourant extends CompteBancaire {
    private String numCarte;  3 usages
    private double decouvertMax;  4 usages

    public CompteCourant(String numero, double solde, String devise, Client titulaire, String numCarte, double decouvertMax) {
        super(numero, solde, devise, titulaire);
        this.numCarte = numCarte;
        this.decouvertMax = decouvertMax;
    }

    public String getNumCarte() {  no usages
        return numCarte;
    }
    public void setNumCarte(String numCarte) {  no usages
        this.numCarte = numCarte;
    }

    public double getDecouvertMax() {  no usages
        return decouvertMax;
    }
    public void setDecouvertMax(double decouvertMax) {  no usages
        this.decouvertMax = decouvertMax;
    }
    public void debiter(double montant) {  1 usage
        if (this.getSolde() - montant >= this.decouvertMax) {
            super.debiter(montant);
        } else {
            System.out.println("Erreur : Découvert autorisé dépassé !");
        }
```

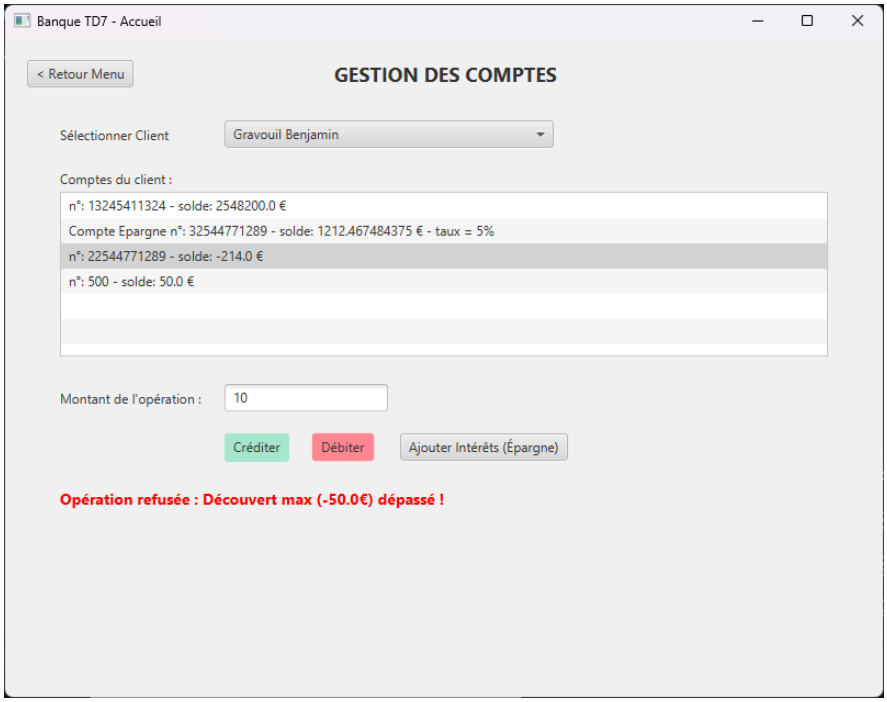
```java
CompteCourant c1 = new CompteCourant( numero: "13245411324",  solde: 2548200,  devise: "€", benjamin,  numCarte: "000000000001",  decouvertMax: -1000);
CompteEpargne c2 = new CompteEpargne( numeroCompte: "32544771289",  solde: 1000,  devise: "€", benjamin,  taux: 0.05);
CompteCourant c3 = new CompteCourant( numero: "22544771289",  solde: -214,  devise: "€", benjamin,  numCarte: "000000000002",  decouvertMax: -50);
```

Exercice 2.14

Création de l’interface graphique (le code est disponible si vous voulez essayer en détail).







Conclusion

Ce TP nous a introduit à l'utilisation de l'héritage en Java.